

The Artifact Promotion Control Model: An Implementation Case Study

Build on Target Machines vs. Build Once and Promote Artifacts

Vasili Gavrilov

Enkli Ylli

First version 3 June 2026; revised 25 August, 2 September and 5 September 2026

Abstract

A previous treatment by the first author [1] presented artifact promotion as a control model for cloud deployment, comparing build-on-target with build-once-and-promote in accessible engineering prose. This paper has two parts. Part I restates the control model in stricter form, covering the release object, the control domains a deployment crosses, the artifact-versus-environment identity distinction, source-control compromise as a control-domain question, and the regulatory frameworks under which the model’s properties become structurally required rather than merely preferable. Part II is an anonymized implementation case study of a production web application on Amazon Web Services, in which the only manual deployment step is the upload of a versioned artifact to an S3 bucket and every subsequent stage runs autonomously. It reports 31 rolling deployments on the production fleet, timed stage by stage from the platform’s own records, and 92 days of production operation under the model: 22 releases built once and promoted in 29 days, the service levels of the deployed system, the resource that bound at deploy time rather than under user load, and the two release units the pipeline did not cover.

Contributions. The parts of the model are each stated elsewhere: the build-once principle in *Continuous Delivery* [2], the build/release/run split in the Twelve-Factor App [4], deployment of approved binaries only in Google’s binary authorization [15, 16], separation as a supply-chain security property [17], and the pipeline-side reading of the frameworks in NIST SP 800-204D [18]. The paper (1) assembles them as one control model, with the control domains named and the promotion boundary as their interface, holding equally for container images, virtual-machine images, operating-system packages, and versioned archives; (2) separates secrets *mechanism* from secrets *timing* and shows that build-time secret injection is incompatible with promotion; (3) frames deployment-time dependency resolution as an adversarial supply-chain surface that scales with the dependency closure; (4) derives a separation of release authority from runtime-secret authority; (5) translates the cited compliance frameworks (FedRAMP/SI-7, SOX 404, FFIEC, DO-178C, FDA 21 CFR Part 11, HIPAA, DoD IL) into the concrete obligations they place on engineers, not only auditors; and (6) reports the production record.

Preface: Why Now

The CVE program published just over 40,000 vulnerabilities in 2024, roughly 38% more than in 2023, the seventh consecutive record year, and submissions have grown fast enough to backlog the National Vulnerability Database [10]. Within that, software supply-chain attacks, which target the build and distribution path rather than a flaw in the running application, are a documented and growing class: the European Union Agency for Cybersecurity reports a marked rise in their number

and sophistication [9], and a peer-reviewed systematization catalogues 107 attack vectors drawn from 94 real incidents [7].

These attacks exploit access to source code or to third-party build inputs at deployment time. In March 2025 a widely consumed continuous-integration action was compromised: its version tags were repointed to code that extracted secrets from the runner process and wrote them, encoded, into workflow logs, where on public repositories anyone could read them [6]. Over twenty thousand repositories depended on it. Under any deployment model where the production execution path consumes source code or third-party build inputs at deployment time, one upstream compromise becomes a production incident on the next rollout.

Where a deployment failure is a safety failure, in aviation control, infusion pumps, medical imaging, industrial process control, the regulatory regimes require deployment evidence that build-on-target models produce only with substantial extra effort, if at all. Part I makes the architectural argument and enumerates the frameworks. Part II documents what the implementation looks like, how long it takes, and what is easy to get wrong.

Part I: The Control Model

1 The Release Object

A common framing of the release problem is the question:

Did we build the latest code on production?

A stricter framing is:

Which approved artifact is currently deployed?

“Latest code” is a property of a version-control repository and changes whenever a commit lands. An *approved artifact* is a concrete object: it has a version label, an integrity hash, a source revision identifier, a build timestamp, an approval record, a deployment history, and a rollback path. Engineers, project managers, testers, auditors, and stakeholders can reason about one object with these attributes; they cannot reason coherently about “the latest code”, because each is looking at a different moving snapshot of it. In this paper the release is the artifact selected for deployment.

2 Two Deployment Models

Environment names vary by organization: development, integration, quality assurance, staging, user acceptance, pre-production, production, disaster recovery, customer replicas. A deployment model that depends on them is not transferable. The model here depends on one property: where in the topology the build occurs.

Model A: build on target. Each deployment instance retrieves the source revision for the intended release, resolves the dependency closure against external sources, builds on the instance, and deploys the locally produced bytes.

Model B: build once and promote. One controlled build produces a deployable artifact identified by a version label and a content hash. The same artifact is stored, identified, approved, and promoted through the environments without rebuild.

```
Model A:
    source repository -> target instance -> local build -> local deploy -> runtime

Model B:
    source repository -> controlled build -> artifact store -> deployment -> runtime
    configuration -> runtime
```

Model A is fine for local development, prototypes, single-machine systems, and short-lived test hosts. Its limitation is that it collapses source checkout, dependency resolution, build, deployment placement, and runtime execution onto each target instance, which makes the separations that controlled release needs operationally difficult. Model B makes the release boundary explicit: the production instance receives a known deployable unit and does not create the release by compiling during deployment.

The models also select an environment differently. Under Model B the environment is determined by which artifact and which runtime configuration are deployed, so one source trunk remains the single source of truth. Under Model A, teams grow per-environment branches to carry the differences, and those branches drift: a fix cherry-picked into one is missed in another, and *what is in production?* becomes a forensic exercise rather than a lookup. The same drift is documented for configuration repositories under GitOps, where a branch per environment is named as an anti-pattern for the same reasons [21].

Model A tendency	Model B
main	main (single source of truth)
cherry-pick (drifts)	
+--> dev branch	one controlled build
+--> qa branch <- picks missed	v
+--> prod branch <- picks missed	artifact-X.Y.zip
	promoted unchanged
"what is in prod?" -> forensic	+--> DEV -> UAT -> PROD
	"what is in prod?" -> an artifact id

Architects often treat the two models as interchangeable plumbing. Building binaries once and deploying the same artifact to every environment is the practice the DevOps Research and Assessment program associates, in its published research, with higher software-delivery performance [3], and the control properties of the rest of Part I depend on the build sitting off the target; they are lost, quietly, the moment it moves on.

3 Control Domains and the Promotion Boundary

Artifact promotion rests on a partition of responsibilities that Model A collapses:

```
Source repository      : code revision history
Build subsystem       : artifact creation
Artifact store        : release identity
Deployment subsystem  : controlled promotion
Runtime environment   : configuration and secrets
Application instance   : execution
```

These are control domains in the security-architecture sense: each has its own ownership, audit trail, and compromise scope. The claim is that source-code evolution and production change are operated in different control domains, and that the promotion boundary is the interface between them. The build subsystem may be a release master’s workstation or a hosted CI service; the model is defined by building the deployable unit once, giving it a stable identity, and promoting it, not by the product.

The model’s parts are established. The binary-repository literature (JFrog Artifactory, Sonatype Nexus) names the verb *promotion* and makes the artifact store the system of record for release identity. *Continuous Delivery* [2] states the principle in its chapter on the deployment pipeline: build the binaries once, deploy the same way to every environment, and check the hash recorded at build time at each stage. The *Twelve-Factor App* [4] canonicalized the build / release / run split. The control-domain reading is Google’s: Binary Authorization for Borg admits to production only binaries whose signed provenance names the reviewed source and the sandboxed build, and the engineer who deploys is not the engineer who wrote the change [15, 16]. Okafor et al. name *separation* as one of three properties a secure supply chain needs, with transparency and validity [17]; the *DevOps Handbook* works the segregation-of-duties case for SOX and PCI through the pipeline [19]; NIST SP 800-204D places the same trust boundaries inside the CI/CD stages [18]; and the integrity mechanisms the boundary depends on, content-addressable identity, signing, and provenance, are the subject of SLSA [11], in-toto [12], Sigstore [13], and reproducible builds [14]. This paper adds nothing to those mechanisms. It draws the boundary they share, states what the two models do to it, and reports one implementation.

4 Artifact Formats and the Layer of Promotion

The artifact format is an implementation choice within the model: a versioned archive deployed onto a prepared instance with a pre-installed runtime, the format of Part II; a container image promoted through a registry; a virtual-machine image promoted through an image repository; an operating-system package (deb, rpm, msi); a language-runtime archive (jar, war, wheel, gem); a filesystem or block-device snapshot. Each draws a different boundary between what is in the artifact and what the host is assumed to have, and the control-domain claims apply to all: each has a stable identity, each can be hashed, signed, scanned, approved, and archived independently of source control, and each promotes without rebuild. The choice turns on artifact size, host-preparation cost, isolation, and the frameworks below, not on the model. A team that already promotes container images through a registry, or archives through an object store, is using the model.

5 Environment-Scoped Runtime Configuration

A second separation, distinct from source/build/artifact, is between artifact identity and environment identity. The source is identical for every environment; the artifact, built bytes and deployment scripts, is identical for every environment; environment-specific runtime data is in neither. That data has two categories. **Secrets:** database passwords, API credentials, encryption keys, OAuth client secrets, mail-transport credentials, whose disclosure causes immediate harm. **Operational configuration:** connection strings, internal service URLs, distribution and bucket identifiers, feature flags, regional identifiers, environment markers, which may not be secret but define the environment the artifact runs in. Both belong to the runtime environment. The implementation rule is that *the arti-*

fact does not know in which environment it is running: it starts on a prepared instance and reads its configuration from one authoritative source, whether a protected file, a configuration-management system, a managed secrets store, or a mounted volume.

A further distinction is between secrets resolved at runtime on the instance and secrets injected into the artifact at build time. Runtime resolution leaves each secret in one place; build-time injection copies it into the artifact, and two copies leak more easily than one. Injection also breaks artifact identity: an artifact carrying one environment’s secrets is no longer the same artifact deployed elsewhere. Whether a secret lives in a protected file or a managed store is an implementation choice of comparable security posture; whether it is resolved at runtime or injected at build time is a model-level decision that determines whether the artifact stays promotable. Part II resolves all secrets at runtime for this reason.

Model A does not logically require secrets in source control, and a disciplined team can run it with runtime configuration correctly; the objection is structural. When every target instance performs checkout, dependency resolution, build-profile selection, compilation, and deployment, the boundaries among source, build configuration, deployment configuration, and runtime configuration are easier to blur, and environment-specific files, branches, profiles, and overrides accumulate near the checkout. The accidental commit is not hypothetical: a longitudinal scan of public GitHub found secrets leaking at thousands per day, and 81% of those found were still present two weeks later [23]; the practices that prevent it are catalogued by Basak et al. [24].

Release authority is separate from runtime-secret authority. Because the artifact contains no secrets, the person who builds and promotes it needs no access to any environment’s secrets. The release master produces an approved artifact and triggers its promotion without holding production credentials; an environment administrator provisions the secrets into the per-environment configuration source. The two are distinct roles, held by different people, and a second person with environment access can roll back or redeploy without the release master. Under Model A, where deploying is itself a build run with the deploying identity’s access, deploying tends to demand the same access as operating the environment.

6 Source-Control Compromise and Control-Domain Separation

If source control is compromised, every model requires incident response; an attacker who can alter source can affect a future build. The difference is the control path from the compromise to a production incident. Under Model A, production instances consume the repository at deployment time, so source control is on the production execution path on every deploy and a repository compromise becomes a production compromise on the next cycle. Under Model B, production receives a previously produced artifact whose identity has been hashed, signed, scanned, approved, and archived. The compromise still affects a future build, but it does not rewrite the identity of an artifact already in production, reveal runtime secrets, or force production to compile from the current repository state. The security property is not that source control cannot be compromised; it is that the compromise has a contained and auditable blast radius rather than an immediate path to runtime execution.

7 Operational Properties

Three properties follow from the artifact-identity boundary, and they are the ones an operations team feels first.

Rollback is artifact selection, not rebuild. Under Model B, reverting a bad release redeploys a previously approved artifact still present, by identity, in the store: the bytes that ran yesterday run again. Under Model A, rollback selects an earlier revision and rebuilds it on every instance, with the same toolchain-drift exposure as the original deployment and no guarantee that today’s rebuild reproduces last week’s bytes. The rebuild does not reproduce even when the recipe is a container file: of 1,096 Docker images rebuilt from their original Dockerfiles in one study, 70 reproduced the same installed package versions and 4 were bitwise identical [25].

Partial-fleet deployment cannot silently split the fleet. In a ten-instance fleet under Model A, each instance checks out, resolves, and builds on its own. If seven succeed and three fail, on a transient network fault, a registry outage, or a local package-state problem, the fleet runs two versions behind one load balancer and no single build log shows it. Under Model B every instance receives the same built artifact; an instance either runs it or does not start. The mixed-version fleet is prevented rather than monitored for.

The production host carries less. Model A puts a source-control client, a runtime sufficient to build, a build tool, and a source checkout on every production instance; each is a patching obligation, an addition to the vulnerability surface, and a disclosure risk if the instance is compromised. Model B needs the application runtime and a deployment procedure. A related availability property follows: under Model A a source-control outage, an expired credential, a branch-protection change, or blocked egress can prevent a deployment; under Model B the approved artifact is already present and installs without reaching any external system.

8 Regulatory Frameworks Under Which the Properties Become Required

The frameworks below are written in the vocabulary of auditors, but the obligations land on the engineers who operate the deployment, so each entry states what the team must be able to show at deploy time and why build-on-target cannot show it. The research literature on continuous delivery in safety-critical settings treats the same requirement from the process side, as the obligation to re-verify every delivered build against the safety analysis [26].

FedRAMP and NIST SP 800-53 control SI-7. FedRAMP authorizes cloud services that handle U.S. federal data on the controls of SP 800-53 [5], and SI-7 (“Software, Firmware, and Information Integrity”) requires detecting unauthorized change to executable code and configuration, in practice by verifying each artifact’s hash at deploy time against the value recorded at build time. Promotion records the hash once, stores it with the release identity, and re-verifies it before the instance accepts the artifact; under Model A each host builds its own bytes from a local toolchain and leaves nothing fixed to verify against.

Sarbanes–Oxley Section 404. Section 404 requires management of public companies to assess internal controls over financial reporting, which drives segregation of duties between code authors and production operators. Granting a production machine the authority to compile arbitrary remote source grants every code author production-execution rights. Promotion makes the separation me-

chanical, the resolution the *DevOps Handbook* reaches for the same controls [19]: the commit, the approved artifact, and the promotion step each carry their own owner and audit trail.

FFIEC IT Examination Handbook. The handbook the U.S. banking regulators examine against expects controlled, auditable change management with development, test, and production separated. When an examiner asks for the change record of what is in production, promotion answers with an artifact identity, an approval, a deploy timestamp, and a rollback record; Model A answers with a history reconstructed from per-machine build logs.

RTCA DO-178C. The civil-aviation software standard mandates traceability from requirements through source and executable object code to the verification record, and certification accepts one specific, frozen executable. A deployment that recompiles on each target ships bytes that were never the verified ones.

FDA 21 CFR Part 11. The rule on electronic records in pharmaceutical, biotech, and medical-device processes requires records of system change that are “trustworthy, reliable, and . . . equivalent to paper records,” which presumes an identifiable change unit. That unit is the artifact with its validation runs executed against those exact bytes; a per-host compile produces bytes no electronic record describes.

HIPAA Security Rule. Its technical safeguards for electronic protected health information include integrity controls, which in deployment reduce to the SI-7 obligation: show that the bytes serving patient data are the approved bytes.

Department of Defense Impact Levels IL-4 to IL-6. Higher impact levels run in enclaves with restricted or no public egress, so a model that checks out source and resolves dependencies at deploy time has nothing to reach; only a self-contained approved artifact crosses the boundary. The Department’s own reference design implements exactly this, a curated registry of approved, hardened container images accepted once and carried across classification levels without rebuild [20].

Promotion does not by itself produce compliance with any of these; compliance is a larger system property. The point of the enumeration is that the boundaries the frameworks require are the boundaries the model establishes, and that a team expecting to be evaluated under any of them is better off drawing them early than under audit pressure. The same byte-identity property is also wanted where no mandate exists: in latency-sensitive systems tuned with specific optimization flags, profile-guided compilation, and pinned dependencies, a per-target rebuild on heterogeneous hosts introduces timing variance between instances nominally running the same release, and promoting one tuned artifact removes it by construction.

9 Common Objections

“Old artifacts ship vulnerabilities.” Old artifacts should not be promoted indefinitely. Vulnerability response is cleaner under promotion: rebuild once, identify, scan, approve, promote. The alternative is coordinating source pulls and local builds across a fleet with no way to verify that all instances built equivalent bytes.

“This is overengineering.” An artifact store, a versioned package, a deployment procedure, and an environment-scoped configuration set establish the boundary. The complexity of larger implementations belongs to the systems they support.

“Source on the server helps debugging.” Production debugging uses logs, metrics, traces,

symbols, source maps, source archives, or controlled diagnostic packages. A source checkout on production is not a release process.

“Promotion means committing to one CI vendor.” The controlled build can run on a workstation, Jenkins, GitHub Actions, GitLab CI, Azure Pipelines, AWS CodeBuild, or anything else controlled. The model is defined by building once, giving the artifact an identity, and promoting it. Part II builds on a workstation and deploys through CodeBuild used as a script runner; nothing in the model requires either.

Part II: Implementation Case Study

10 Setting

The system is a production web application: a backend exposing a REST API, a static frontend served from a content-delivery network, and a relational database in a managed service. The backend language is immaterial to what follows. The fleet is stateless application instances behind an application load balancer; version control is a single shared trunk with annotated release tags. The infrastructure is on Amazon Web Services, and the specific services (S3, EventBridge, CodeBuild, Elastic Beanstalk, CloudFront) are named because a recipe is useful only if concrete; equivalents exist elsewhere. Withheld is anything that would grant access or identify the deployment: account identifiers, IAM principals, bucket names, distribution identifiers, hostnames.

The pipeline was built on a development environment in June 2026 and stood up for user-acceptance testing and production on 26 and 29 July 2026; the development environment was retired on 11 August. The production fleet is two instances, one per availability zone, deployed in rolling batches of one. The rolling-deployment timings in Results are from the production fleet; the operating figures are from the environment that carried the production traffic in the window, two instances against the smallest burstable database class.

11 Materials and Methods

11.1 The deployment pipeline

A release is one command on the release master’s machine: a release build that produces a versioned zip, then an upload of that zip to the environment’s S3 bucket. The upload is the only manual action.

```
Release master (local):
  release build produces artifact-<VER>.zip (built locally; no cloud build)
  aws s3 cp      artifact-<VER>.zip -> s3://<artifact-bucket-for-env>
  |
  | S3 ObjectCreated event
  v
Amazon EventBridge rule (one per environment)
  | fires StartBuild on the CodeBuild project
  v
AWS CodeBuild project (source = NO_SOURCE)
  | downloads the artifact from S3
```

```

    | runs the deployment script (buildspec):
    |   - sync static assets to the CloudFront origin bucket
    |   - create a CloudFront invalidation
    |   - assemble the Elastic Beanstalk source bundle from the artifact
    |   - upload bundle, CreateApplicationVersion, UpdateEnvironment
    v
AWS Elastic Beanstalk
    | rolls each EC2 instance to the new application version
    v
Per EC2 instance, at predeploy (.platform hook):
    | the hook probes each candidate config bucket in S3;
    |   the instance role's IAM policy grants s3:GetObject on
    |   exactly one bucket - the one for this instance's environment
    | pulls the config files to local paths, sets owner and mode
    v
Application restarts and reads its config on startup

```

The release build records the artifact's identity, a checksum and a build time, so every promoted zip carries the hash and timestamp named in the Release Object section. That value, recorded at build time, is what an instance verifies against at deploy time and what identifies the artifact through promotion and rollback, the concrete form of the SI-7 requirement. For tamper evidence the hash should be collision-resistant; a faster checksum still catches corruption in transit.

11.2 CodeBuild runs the deployment and builds nothing

The CodeBuild project's source is `NO_SOURCE`, and the product name works against the reader: no compilation occurs in this stage. The artifact was built before the upload. CodeBuild is an event-triggered, IAM-scoped script runner that downloads the artifact and executes the deployment script against the control plane; it checks out no source and resolves no dependencies. Engineers reading the architecture for the first time assume the build happens here and look in the wrong place when diagnosing a failure, so documentation of this pattern should say on first mention that CodeBuild is the trigger-and-deploy step.

11.3 Dependency materialization

The artifact contains every third-party library the application runs with; nothing is resolved at deployment or at instance start against any hosted registry. Resolving transitive dependencies at deploy time has two failure modes that need no adversary: a maintainer withdraws a published version and a tree that resolved last week no longer does, and two builds of the same revision weeks apart select different transitive versions and produce different bytes. The third is adversarial. Every public registry is a code-injection surface, and the risk scales with the closure: installing an average npm package implicitly trusts about 80 others [22], and malicious packages are a recurring, catalogued class of attack [8, 7]. Under Model A that resolution happens at deploy time, on or near production, on every rollout. Under Model B it happens once, at build time, away from production; the resolved set is captured in the artifact, can be scanned and pinned before approval, and is never re-fetched. The cost is a larger artifact and no free upstream patches; the gain is byte-deterministic builds from a revision and immunity to upstream removals.

11.4 Environment identity via IAM scoping

Each instance must learn which environment it is in so as to fetch the right configuration. An operating-system variable would be a second source of truth that can drift, plus a templating step per deploy; a managed secrets store adds latency and a credential lifecycle to learn one string. Instead, each environment’s instance role is granted `s3:GetObject` on exactly one configuration bucket. The predeploy hook tries each candidate bucket; IAM lets exactly one succeed, and that bucket is the environment. As defense in depth the fetched configuration carries an explicit environment marker, the hook cross-checks it against the bucket name, and on a mismatch the instance refuses to start.

11.5 Per-environment configuration

The per-environment configuration is a few files: application properties, logging, and the application server’s own configuration. They are uploaded once per environment from a secured machine and never written by the pipeline. The artifact is bit-identical across environments; the environment-specific data, including database and mail-transport credentials, lives only in the configuration buckets under encryption at rest and access control. The same keys appear in every environment with different values. Because the artifact carries no secrets and the buckets are written out of band, triggering a deployment needs only the right to upload an artifact, so the role that releases and the role that holds production secrets need not be the same person. A managed secrets service is an equally valid store.

12 Results

The application is about 121,000 non-blank lines of backend source and 88,000 of frontend source; the artifact is 10 MB.

12.1 The first autonomous deployment

The first autonomous end-to-end deployment, on a single-instance development environment, took about 90 seconds from upload to the running application reporting the new version on its status endpoint: about 5 s for the S3 event and CodeBuild startup, 30 s for the runner’s work (artifact download, static sync, CDN invalidation, bundle assembly, version creation, environment update), 28 s for the single-instance rollout, and 5 s for the health check and verification.

12.2 Rolling deployment on the production fleet

The runner and the platform each keep their own timestamps, so every production deployment can be timed after the fact: the runner’s build history gives its start and end, and the platform’s event log gives the start of the environment update, each instance’s completion, and the moment the new version is reported deployed. The table pairs them for the 31 successful rollouts to the two-instance production fleet between 28 July and 3 September 2026, deployed in two batches of one with a health-check interval after each.

Stage	median	range
Runner start to environment update starting (static sync, CDN invalidation, bundle assembly, version creation)	27 s	11 to 39 s
First instance: new version installed and reported	36 s	36 to 38 s
Second instance, from the first instance's completion (health gating, then the same installation)	170 s	150 to 172 s
From the second instance's completion to the platform reporting the new version deployed	138 s	127 to 149 s
Environment update, start to reported deployed	344 s	319 to 360 s
Runner, start to end	398 s	352 to 411 s

The 26 rollouts on the other environment differ only in the first row, 72 s against 27 s, from a larger static synchronization; the platform stages match to within a few seconds. The artifact's installation on an instance is 36 seconds and does not vary; the other five minutes are the platform's rolling policy, and a faster rollout comes from the batch size and the health interval, not from the artifact. The spread is 15% of the median end to end, so a deploy that runs long is a signal in itself.

The record also holds the fleet property claimed in Part I. The first production rollout, on 27 July, failed on one of the two instances; the platform aborted the update, kept the running version on the whole fleet, and the second attempt five minutes later succeeded. Three later runner executions failed within 20 seconds without touching the fleet, because a second artifact was uploaded while a rollout was still in progress and the platform refused the update until the environment was ready; the remedy was to upload again after completion.

12.3 Release cadence

The project carries 63 tagged releases since its first in March 2026. In the 29 days from 27 July to 24 August 2026, the first month in which both remaining environments ran the pipeline, 22 were built once on the release master's machine and uploaded. A promotion to the next environment is the upload of the same zip to that environment's bucket; nothing is rebuilt and no second tag is cut. The platform's version label joins the artifact version to the runner's build number, so two uploads of one artifact are distinguishable in the history while the bytes are not.

12.4 Production operation

The figures below are from 92 days of access logs and platform metrics on the environment carrying the production traffic, taken on 1 August 2026. The first row is why the others need a filter: health checks and scanner probes are 96% of the log, and a latency or error figure over the unfiltered log describes the load balancer.

Requests in 92 days	504,616 health checks, about 40,000 scanner probes, 20,851 API requests
Latency, API requests	p95 64 ms, p99 316 to 392 ms
Errors	28 HTTP 500 in 20,851 requests, 0.13%
Largest load carried	124 concurrent users, 15.6% peak database CPU, 7 of 60 connections

The database is the smallest burstable class the platform offers, so these are a floor, not a capacity. The resource that binds first is connections, and it binds on fleet size rather than on users, because each instance holds its own pool:

```
instances x pool size + 1 worker + 3 internal < 0.8 x max_connections
```

On the measured environment it stood at 48 of 48. A rolling deploy, and a rollback, is the one moment when two batches of instances hold pools at once, so pool exhaustion appears during a deploy rather than under user load. The budget has to be sized for both batches, and when it is short the fix is the database class, since an added instance consumes the scarce resource rather than relieving it.

12.5 What the promotion boundary did not cover

A release of this system is three units: the artifact, the schema migrations it may carry, and a worker process that runs as a separate JVM on a persistent host outside the fleet. The pipeline promoted the first. The other two were runbook steps, and an application deploy left the worker running the previous release's code until someone restarted it. The failure is silent: the old worker admits or refuses work by the previous release's rules, and the symptom is that nothing happens, not an error. On 19 August 2026 the release command was extended to drive all three units: it uploads the artifact, applies pending migrations only if the environment's recorded schema version is behind the repository's, after a database snapshot on production, and repoints and restarts the worker on every release, unconditionally, because the alternative is a hand-kept list of what the worker loads, and such a list rots.

The second hazard is of the same kind. The configuration is pulled by the predeploy hook at instance launch; editing the bucket changes nothing running, and restarting the application server does not re-run the hook. A configuration change reaches the fleet only through an instance refresh.

13 Discussion

The two models diverge at fleet scale and on rollback. Under Model A the build and dependency resolution are paid once per instance and grow with the fleet; under Model B the build is paid once. Across 31 rollouts the artifact's installation on an instance is a constant 36 seconds. Under Model A that step would also carry a checkout, a resolution, and a compilation per instance, with their variance, while the platform's five minutes of health gating around it would be unchanged: the rollout time is a property of the rolling policy, and the model's term in it is fixed and small.

A rollback here is the same operation as a deploy. The promotion command uploads the previously promoted artifact, the runner and the platform run the same stages, static assets included, and the version label records a lower number, so the table is also its duration and it draws on the same connection budget. No rollback was needed in the window. The schema does not roll back with the artifact; its rollback is manual, which is the usual discipline. The release command takes a named database snapshot before it migrates a shared environment and refuses to migrate one it cannot snapshot; a restore creates a new database instance rather than overwriting the old, and the configuration is repointed at it. The artifact rolls back through the pipeline in six minutes; the data rolls back by hand, from a snapshot whose existence the pipeline guaranteed.

Two of the production figures are explained by something other than the deployment model. The application is on the plain side of the pairs measured in the first author’s tokens-to-trace study [27]: servlets over JDBC with hand-written SQL, a static frontend without a framework, no code generation and no build-time dependency resolution. That discipline is out of scope here. It accounts for the latency row, where a request passes through nothing between the servlet and the database, and for the pace: about 209,000 non-blank lines and 63 releases in ten months from the first commit, by four contributors. The deployment model made each release a single upload; it did not make them fast to write.

The finding the authors would state first to a team adopting the model is the uncovered release units: draw the promotion boundary around everything a release changes, and treat a unit the pipeline does not carry as a half release until it does.

14 Limitations

One application on one cloud platform; multi-region rollouts, blue-green and canary overlays, and stateful-workload migrations are out of scope. No rollback occurred in the window. The deployment timings are from two-instance fleets under one rolling policy; a larger fleet or another batch size changes the platform’s share of the time, not the artifact’s. The operating figures were taken on the smallest burstable database class and bound the model from below. Schema migrations run from the release command outside the autonomous pipeline; carrying them inside it raises correctness questions not addressed here. The regulatory section names the regimes and does not attempt the control-by-control mapping a compliance package requires.

15 Conclusion

A pipeline whose only manual step is the upload of a versioned artifact can be operated reliably under two conditions: the environment identity is carried by IAM scoping rather than by a mutable side channel, and each instance pulls its configuration at start from one environment-scoped source. Part I argued that artifact promotion is the right control model. Part II reports that the mechanism is small, that it carried 22 releases in its first month with a 36-second installation per instance inside a six-minute rolling policy, and that the two things it got wrong were both outside the boundary it drew: a release unit the pipeline did not promote, and a configuration change that reaches a running fleet only by replacing it.

Author Note

The first author’s prior work spans the regulated domains treated here. In a Canadian provincial health system he produced and distributed hashable binary packages, in C and Java, to hospitals, health-care facilities, and medical-education institutions under the province’s health-information-protection law and federal privacy legislation, the Canadian counterparts of the HIPAA Security Rule, which made a fixed, hashable binary a precondition for distribution. He also built financial rule engines, among them a provincial consolidation of financial and statistical accounts that ranked first among provinces for data quality.

The second author implemented the deployment infrastructure of Part II: the event-triggered runner, the managed platform environment, and the IAM scoping. He holds a PhD and is a certified cloud- and information-security practitioner (CISSP, CCSP).

References

- [1] Gavrilov, V. (2026). *Artifact Promotion as a Control Model for Stable Cloud Deployment: Build on Target Machines vs. Build Once and Promote Artifacts*. Zenodo. <https://doi.org/10.5281/zenodo.20451077>
- [2] Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- [3] Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- [4] Wiggins, A. (2017). *The Twelve-Factor App*. Heroku. <https://12factor.net/>
- [5] National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations*. NIST Special Publication 800-53, Revision 5.
- [6] Cybersecurity and Infrastructure Security Agency (CISA). (2025). *Supply Chain Compromise of Third-Party tjs-actions/changed-files (CVE-2025-30066) and reviewdog/action-setup (CVE-2025-30154)*. CISA Alert, 18 March 2025. GitHub Advisory GHSA-mrrh-fwg8-r2c3 (CVE-2025-30066), <https://github.com/advisories/GHSA-mrrh-fwg8-r2c3>.
- [7] Ladisa, P., Plate, H., Martinez, M., & Barais, O. (2023). SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. <https://doi.org/10.1109/SP46215.2023.10179304>
- [8] Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*, LNCS 12223. Springer.
- [9] European Union Agency for Cybersecurity (ENISA). (2021). *Threat Landscape for Supply Chain Attacks*.
- [10] CVE Program and National Vulnerability Database. *Published CVE Record Counts by Year*. <https://www.cve.org/about/Metrics>
- [11] Open Source Security Foundation (OpenSSF). (2023). *Supply-chain Levels for Software Artifacts (SLSA), Version 1.0*. <https://slsa.dev/>
- [12] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., & Cappos, J. (2019). in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*.
- [13] Newman, Z., Meyers, J. S., & Torres-Arias, S. (2022). Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. <https://doi.org/10.1145/3548606.3560596>
- [14] Reproducible Builds Project. *Reproducible Builds: An Independently-Verifiable Path from Source to Binary Code*. <https://reproducible-builds.org/>
- [15] Google Cloud. (2019). *Binary Authorization for Borg: How Google Verifies Code Provenance and Implements Code Identity*. <https://cloud.google.com/docs/security/binary-authorization-for-borg>
- [16] Adkins, H., Beyer, B., Blankinship, P., Lewandowski, P., Oprea, A., & Stubblefield, A. (2020). *Building Secure and Reliable Systems*, chapter 14, Deploying Code. O’Reilly.
- [17] Okafor, C., Schorlemmer, T. R., Torres-Arias, S., & Davis, J. C. (2022). SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED)*. <https://doi.org/10.1145/3560835.3564556>

- [18] Chandramouli, R., Kautz, F., & Torres-Arias, S. (2024). *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. NIST Special Publication 800-204D. <https://doi.org/10.6028/NIST.SP.800-204D>
- [19] Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*, chapter 23, Protecting the Deployment Pipeline and Integrating into Change Management and Other Security and Compliance Controls. IT Revolution Press.
- [20] U.S. Department of Defense, Chief Information Officer. *DoD Enterprise DevSecOps Fundamentals*, v2.5, and *DoD Enterprise DevSecOps Reference Design: CNCF Kubernetes* (2021). <https://dodcio.defense.gov/Portals/0/Documents/Library/DoD%20Enterprise%20DevSecOps%20Fundamentals%20v2.5.pdf>
- [21] Kapelonis, K. (2022). *Stop Using Branches for Deploying to Different GitOps Environments*. Codefresh. <https://codefresh.io/blog/stop-using-branches-deploying-different-gitops-environments/>
- [22] Zimmermann, M., Staicu, C.-A., Tenny, C., & Pradel, M. (2019). Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *28th USENIX Security Symposium*.
- [23] Meli, M., McNiece, M. R., & Reaves, B. (2019). How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories. In *Network and Distributed System Security Symposium (NDSS)*.
- [24] Basak, S. K., Neil, L., Reaves, B., & Williams, L. (2022). What are the Practices for Secret Management in Software Artifacts? In *2022 IEEE Secure Development Conference (SecDev)*. <https://arxiv.org/abs/2208.11280>
- [25] Malka, J., Zacchiroli, S., & Zimmermann, T. (2026). *Docker Does Not Guarantee Reproducibility*. arXiv:2601.12811. <https://arxiv.org/abs/2601.12811>
- [26] Vöst, S., & Wagner, S. (2017). Keeping Continuous Deliveries Safe. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 259–261. <https://doi.org/10.1109/ICSE-C.2017.61>
- [27] Gavrilov, V. (2026). *When Abstraction Becomes Indirection: Tokens-to-trace, measured on six stacks*. Zenodo. <https://doi.org/10.5281/zenodo.22113993>